UNIVERSIDADE FEDERAL DE SERGIPE

Programação dinâmica

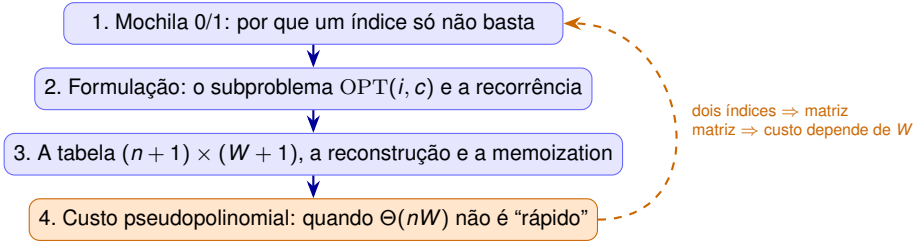
O problema da mochila

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

15 de outubro de 2026

O que você vai aprender hoje



Na aula anterior, $\mathrm{OPT}(j)$ tinha **um** índice. Hoje a última decisão muda **dois** parâmetros: quais itens restam e quanto cabe.

Mochila 0/1

Por que precisamos de um algoritmo para a mochila?

Problema:

Há n itens, cada um com **peso** w_i e **valor** v_i, e uma mochila que aguenta peso total W. Quais itens levar para maximizar o valor?

Aparece como orçamento de projetos (custo $\times$ retorno), carga de caminhões e contêineres, escolha de tarefas para uma janela de CPU.

A força bruta testa 2^n subconjuntos: com $n = 60$, $\approx 10^{18}$.

Solução:

Programação dinâmica, com os mesmos quatro passos da aula anterior: estrutura, recorrência, tabela, reconstrução.

A novidade: o subproblema precisa lembrar **quais itens** ainda podem ser usados e **quanta capacidade** sobrou. A tabela vira uma matriz $(n+1) \times (W+1)$ e o tempo fica $\Theta(nW)$.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Levitin (2012), §8.2 [9].

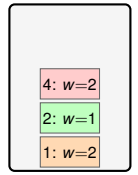
Mochila 0/1: definição

Definição

Dados pesos $w_1, \ldots, w_n$, valores $v_1, \ldots, v_n$ e capacidade W (inteiros positivos), encontrar $S \subseteq \{1, \ldots, n\}$ que

$$\text{maximiza} \sum_{i \in S} v_i \quad \text{sujeito a} \quad \sum_{i \in S} w_i \leq W.$$

Soma de subconjuntos (*subset sum*): o caso $v_i = w_i$; encher a mochila o máximo possível.



fora:
item 3 (w=3, v=20)

dentro: peso 5,
valor $12 + 10 + 15 = 37$

Intuição

"0/1": cada item entra inteiro ou não entra. Não dá para levar meio item, nem duas cópias do mesmo.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Dantzig (1957) [3]; Feofiloff, *Mochila booleana* [5].

Exemplo: o guloso por valor/peso falha na mochila 0/1



Escolher pela maior razão v_i/w_i é **ótimo** na mochila *fracionária*, mas na 0/1 sobra espaço que nenhum item restante preenche bem. Precisamos considerar as duas opções para cada item.

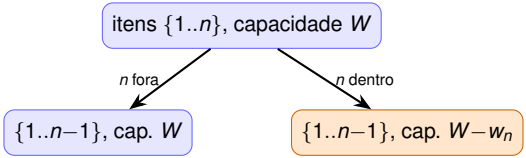
Para aprofundar: Cormen et al. (2009), §16.2, Figura 16.2 [1].

Tentativa: um índice só, como nos intervalos

Seja $\mathcal{O}$ uma solução ótima e $\mathrm{OPT}(j)$ o melhor valor com os itens $\{1, \ldots, j\}$. Olhe para a **última decisão**, o item n:

- $n \notin \mathcal{O}$: então $\mathrm{OPT}(n) = \mathrm{OPT}(n-1)$. ✓
- $n \in \mathcal{O}$: sobram os itens $\{1, \ldots, n-1\}$, mas agora só cabem $W - w_n$. $\mathrm{OPT}(n-1)$ foi calculado para capacidade W: não serve.

Nos intervalos, aceitar *j* só *eliminava* itens (os incompatíveis). Aqui, aceitar *n* não elimina ninguém: muda a **capacidade**.



subproblema **novo**: precisamos de um para cada capacidade $0, \ldots, W$

Para aprofundar: Kleinberg e Tardos (2006), §6.4, “Designing the algorithm: a false start” [8].

Recapitulando: mochila 0/1

O que aprendemos

- Mochila 0/1: escolher itens de valor máximo com peso total $\leq W$; soma de subconjuntos é o caso $v_i = w_i$.
- A força bruta testa 2^n subconjuntos.
- O guloso por v_i/w_i resolve a versão fracionária, mas falha na 0/1 (160 contra 220 no exemplo do Cormen).
- Um índice só não basta: pôr o item n na mochila reduz a capacidade, então o subproblema precisa de um segundo parâmetro.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Cormen et al. (2009), §16.2 [1].

Formulação

Por que dois índices resolvem?

Problema:

A decisão sobre o item i deixa um problema do *mesmo tipo*, com menos itens e, possivelmente, menos capacidade. Com um índice só, não há como nomear esse problema menor.

Solução:

Indexar o subproblema pelos dois parâmetros que mudam:

- i: só os itens $1, \ldots, i$ estão disponíveis;
- c: a capacidade disponível é c, com $0 \leq c \leq W$.

São $(n+1)(W+1)$ subproblemas, cada um resolvido em $O(1)$ a partir de dois subproblemas da linha $i-1$.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Levitin (2012), §8.2 [9].

O subproblema $\mathrm{OPT}(i, c)$ e a recorrência

Definição

$\mathrm{OPT}(i, c)$ = maior valor de um subconjunto de $\{1, \ldots, i\}$ com peso total $\leq c$. A resposta do problema é $\mathrm{OPT}(n, W)$.

$$\mathrm{OPT}(i, c) = \begin{cases} 0 & \text{se } i = 0, \\ \mathrm{OPT}(i-1, c) & \text{se } w_i > c \quad \text{(o item } i \text{ não cabe)}, \\ \max\big(\underbrace{\mathrm{OPT}(i-1, c)}_{i \text{ fora}},\ \underbrace{v_i + \mathrm{OPT}(i-1,\ c - w_i)}_{i \text{ dentro}}\big) & \text{caso contrário.} \end{cases}$$

Intuição

Para cada item, as duas opções são testadas, e é isso que o guloso não fazia. A tabela evita repetir o cálculo de cada combinação (itens, capacidade).

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Levitin (2012), §8.2 [9].

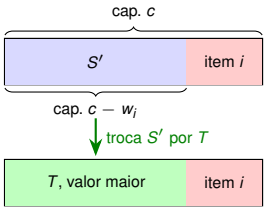
Subestrutura ótima: recorta e cola

Seja S ótimo para (i, c), com $i \in S$. Afirmação:
$S' = S \setminus \{i\}$ é ótimo para $(i-1,\ c-w_i)$.

Prova. S' é viável: pesa $\leq c - w_i$. Se existisse T viável para $(i-1, c-w_i)$ com valor maior que S', então $T \cup \{i\}$

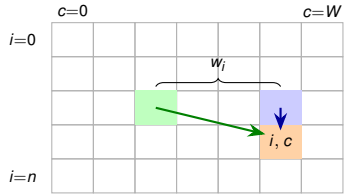
- pesa $\leq (c-w_i) + w_i = c$ (viável para (i,c)), e
- vale $v(T) + v_i > v(S') + v_i = v(S)$,

contradizendo a otimalidade de S. O caso $i \notin S$ é análogo, com $(i-1, c)$. □



Para aprofundar: Cormen et al. (2009), §16.2 (subestrutura ótima da mochila 0/1) e §15.3 [1].

O grafo de subproblemas e a ordem de preenchimento



A célula (i, c) depende só da **linha de cima**:

- $(i-1, c)$: o item i fica fora;
- $(i-1, c-w_i)$: o item i entra.

O grafo de subproblemas é um DAG; preencher **linha por linha** ($i = 0, 1, \ldots, n$) respeita todas as dependências. Dentro da linha, qualquer ordem de c serve.

Tempo: $(n+1)(W+1)$ células $\times$ $O(1)$ por célula $= \Theta(nW)$.
Espaço: $\Theta(nW)$ para a matriz.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Cormen et al. (2009), §15.1, “Subproblem graphs” [1].

Recapitulando: formulação

O que aprendemos

- Subproblema com dois índices: $\mathrm{OPT}(i, c)$, os itens $1..i$ com capacidade c.
- Recorrência: $\mathrm{OPT}(i, c) = \max(\mathrm{OPT}(i-1, c),\ v_i + \mathrm{OPT}(i-1, c-w_i))$, com o segundo termo só quando $w_i \leq c$.
- A subestrutura ótima sai do recorta e cola, exatamente como em intervalos.
- Cada célula depende da linha anterior; linha por linha dá $\Theta(nW)$ de tempo e de espaço.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8].

Tabela e reconstrução

Bottom-up em C

Instância condutora ($W = 5$):

item i	peso w_i	valor v_i
1	2	12
2	1	10
3	3	20
4	2	15

O guloso por v_i/w_i acerta nesta instância (itens 2, 4, 1); a PD acerta *sempre*.

```
/* M[i][c]: melhor valor com itens 1..i, capacidade c */
int mochila( int n, int W, int w[], int v[]) {
   for (int c = 0; c <= W; ++c) M[0][c] = 0;
   for (int i = 1; i <= n; ++i)
      for (int c = 0; c <= W; ++c) {
         M[i][c] = M[i-1][c];                /* i fora   */
         if (w[i] <= c && v[i] + M[i-1][c-w[i]] > M[i][c])
            M[i][c] = v[i] + M[i-1][c-w[i]]; /* i dentro */
      }
   return M[n][W];
}
```

Para aprofundar: Levitin (2012), §8.2 [9]; Kleinberg e Tardos (2006), §6.4, SUBSET-SUM(n, W) [8].

Exemplo: a tabela M preenchida

item (w_i, v_i)	c=0	c=1	c=2	c=3	c=4	c=5
i=0	0	0	0	0	0	0
1 (2, 12)	0	0	12	12	12	12
2 (1, 10)	0	10	12	22	22	22
3 (3, 20)	0	10	12	22	30	32
4 (2, 15)	0	10	15	25	30	37

Para aprofundar: Levitin (2012), §8.2 (mesma instância) [9].

Célula $(4,5)$, item 4 com $w_4 = 2$, $v_4 = 15$:

$$\max(\underbrace{32}_{M[3][5]},\ 15 + \underbrace{22}_{M[3][3]}) = 37.$$

Linha 1: só o item 1 ($w = 2$) $\Rightarrow$ 12 a partir de $c = 2$.

Linha 3, $c = 4$:
$\max(22,\ 20 + M[2][1]) = \max(22, 30) = 30$.

Resposta: $\mathrm{OPT}(4,5) = M[4][5] = 37$.

Reconstruindo a solução

	c=0	c=1	c=2	c=3	c=4	c=5
i=0	0	0	0	0	0	0
1 (2, 12)	0	0	12	12	12	12
2 (1, 10)	0	10	12	22	22	22
3 (3, 20)	0	10	12	22	30	32
4 (2, 15)	0	10	15	25	30	37

```
int c = W;
for (int i = n; i >= 1; --i)
   if (M[i][c] != M[i-1][c]) {  /* i entrou */
      printf( " %d", i);
      c -= w[i];
   }
```

$M[4][5] = 37 \neq 32$: **4 entra**, $c = 3$.
$M[3][3] = 22 = M[2][3]$: 3 fica fora.
$M[2][3] = 22 \neq 12$: **2 entra**, $c = 2$.
$M[1][2] = 12 \neq 0$: **1 entra**, $c = 0$.
Solução $\{1, 2, 4\}$, peso 5, valor 37. Custo $O(n)$.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Levitin (2012), §8.2 [9].

Memoization: só as células necessárias

```c
/* M[i][c] == -1: ainda nao calculado */
int memo( int i, int c) {
   if (i == 0) return 0;
   if (M[i][c] < 0) {
      int r = memo( i-1, c);
      if (w[i] <= c && v[i] + memo( i-1, c-w[i]) > r)
         r = v[i] + memo( i-1, c-w[i]);
      M[i][c] = r;
   }
   return M[i][c];
}
```

A mesma recorrência, de cima para baixo, a partir de (n, W).

	c=0	c=1	c=2	c=3	c=4	c=5
i=0	0	0	0	0	0	0
1 (2, 12)	0	0	12	12	12	12
2 (1, 10)	0	–	12	22	–	22
3 (3, 20)	–	–	–	22	–	32
4 (2, 15)	–	–	–	–	–	37

Só **13 das 24** células com $i \geq 1$ são calculadas: as alcançáveis a partir de $(4, 5)$ no grafo de subproblemas (a linha $i = 0$ é caso base). Com pesos grandes e W grande, a economia cresce; o pior caso continua $\Theta(nW)$.

Para aprofundar: Levitin (2012), §8.2, “Memory functions” [9].

Recapitulando: tabela e reconstrução

O que aprendemos

- Bottom-up: dois laços aninhados, linha i calculada a partir da linha $i-1$; $\Theta(nW)$.
- Na instância condutora, $M[4][5] = 37$ com a solução $\{1, 2, 4\}$.
- Reconstrução em $O(n)$: se $M[i][c] \neq M[i-1][c]$, o item i entrou e c diminui de w_i.
- Memoization calcula só as células alcançáveis a partir de (n, W) (13 de 24 no exemplo).

Para aprofundar: Levitin (2012), §8.2 [9]; Kleinberg e Tardos (2006), §6.4 [8].

Custo pseudopolinomial

$\Theta(nW)$ é um algoritmo polinomial?

Problema:

Com $n = 100$ itens e $W = 10^{18}$ (pesos em miligramas, valores em centavos...), a tabela teria 10^{20} células. A entrada, porém, cabe em poucos quilobytes: W ocupa só 60 bits.

O tempo cresce com o **valor** de W, não com o **número de bits** usados para escrevê-lo.

Solução: medir corretamente.

O tamanho da entrada é $N \approx \log_2 W + \sum_i (\log_2 w_i + \log_2 v_i)$ bits. Se $W \approx 2^k$, o algoritmo gasta $n \cdot 2^k$: **exponencial em** k.

Algoritmos assim se chamam **pseudopolinomiais**: ótimos quando W é pequeno, inviáveis quando W é grande.

Para aprofundar: Kleinberg e Tardos (2006), §6.4 [8]; Garey e Johnson (1979), §4.2 [6].

Tempo pseudopolinomial: definição

Definição

Um algoritmo é pseudopolinomial se seu tempo é limitado por um polinômio no tamanho da entrada **e** na **magnitude** do maior número da entrada (aqui, W), e não só no tamanho em bits.

Intuição

Escrever W em **unário** (cinco como 11111) tornaria $\Theta(nW)$ polinomial. Em binário, cada bit a mais em W **dobra** o tempo.

$n = 100$ itens, $W \approx 2^k$:

bits k	W	células nW
10	$\approx 10^3$	10^5
20	$\approx 10^6$	10^8
30	$\approx 10^9$	10^{11}
40	$\approx 10^{12}$	10^{14}
64	$\approx 1{,}8 \cdot 10^{19}$	$1{,}8 \cdot 10^{21}$

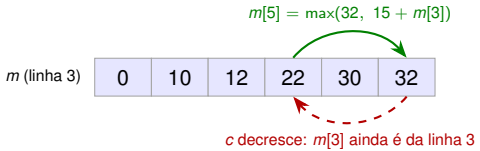
Somar 10 bits a W multiplica o tempo por ≈ 1000.

Para aprofundar: Garey e Johnson (1979), §4.2 [6]; Kleinberg e Tardos (2006), §6.4 [8].

Economia de espaço: uma linha só

```
/* m[c] guarda a linha atual da tabela */
for (int c = 0; c <= W; ++c) m[c] = 0;
for (int i = 1; i <= n; ++i)
   for (int c = W; c >= w[i]; --c)  /* decrescente! */
      if (v[i] + m[c-w[i]] > m[c])
         m[c] = v[i] + m[c-w[i]];
return m[W];
```

Espaço $\Theta(W)$ em vez de $\Theta(nW)$; o tempo continua $\Theta(nW)$. Sem a matriz, porém, a reconstrução da página anterior não funciona.

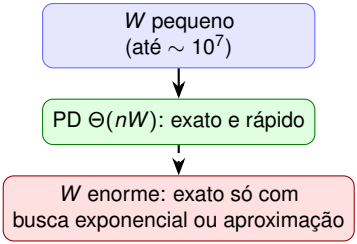


Processando c de W para baixo, $m[c - w_i]$ ainda tem o valor da linha $i - 1$ quando é lido. Em ordem **crescente**, ele já teria sido atualizado e o item i poderia entrar **duas vezes**. Isso é outro problema (o desafio do tutorial).

Para aprofundar: Dasgupta et al. (2008), §6.4 [4]; Feofiloff, *Mochila booleana* [5].

A mochila é NP-difícil

- A versão de decisão (“existe S com peso $\leq W$ e valor $\geq K$?”) está na lista de 21 problemas NP-completos de Karp (1972).
- Soma de subconjuntos é NP-completo: Cormen et al., §34.5.5.
- Não se conhece algoritmo **polinomial em** N; se existisse, teríamos $\mathrm{P} = \mathrm{NP}$.
- O $\Theta(nW)$ não contradiz nada disso, porque é pseudopolinomial. Problemas assim são chamados **fracamente** NP-completos.



NP-completude e reduções são o assunto da Unidade 3.

Para aprofundar: Karp (1972) [7]; Cormen et al. (2009), §34.5.5 [1]; Garey e Johnson (1979), §4.2 [6].

Recapitulando: custo pseudopolinomial

O que aprendemos

- $\Theta(nW)$ é polinomial no *valor* de W, mas exponencial no número de bits de W: o algoritmo é **pseudopolinomial**.
- Na prática: excelente para W até alguns milhões, inviável para $W \sim 10^{18}$.
- Uma linha só basta para o valor: espaço $\Theta(W)$, com c **decrescente**.
- A mochila 0/1 é NP-difícil; a PD não contradiz isso.

Para aprofundar: Garey e Johnson (1979), §4.2 [6]; Dasgupta et al. (2008), §6.4 [4].

Fechando o ciclo

	intervalos ponderados	mochila 0/1	próxima: LCS
subproblema	$\mathrm{OPT}(j)$	$\mathrm{OPT}(i, c)$	$\mathrm{OPT}(i, j)$
última decisão	j entra ou não	i entra ou não	casa x_i com y_j?
tabela	vetor $n + 1$	matriz $(n+1) \times (W+1)$	matriz $(m+1) \times (n+1)$
tempo	$O(n \log n)$	$\Theta(nW)$, pseudopolinomial	$\Theta(mn)$

Próxima aula: maior subsequência comum e distância de edição. Também haverá dois índices, agora as posições em duas sequências, e o tempo será polinomial de verdade.

Para estudar: Kleinberg e Tardos, §6.4; Levitin, §8.2 (exercícios); a mochila *com repetição* (Dasgupta, §6.4) é o desafio do tutorial.

Para aprofundar: Kleinberg e Tardos (2006), §§6.4 e 6.6 [8]; Dasgupta et al. (2008), §6.4 [4]; Cormen et al. (2009), §15.4 [1].

Referências I



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



George B. Dantzig.
Discrete-variable extremum problems.
Operations Research, 5(2):266–288, 1957.



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.
Algorithms.
McGraw-Hill, 2008.
Seção 2.1: Multiplication (o truque de Gauss).



Paulo Feofiloff.
Mochila booleana.
Análise de Algoritmos, IME-USP, 2015.
Acesso em: 10 out. 2026.

Referências II



Michael R. Garey and David S. Johnson.

Computers and Intractability: A Guide to the Theory of NP-Completeness.
W. H. Freeman, 1979.



Richard M. Karp.

Reducibility among combinatorial problems.
In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York, 1972.



Jon Kleinberg and Éva Tardos.

Algorithm Design.
Pearson/Addison-Wesley, 2006.
Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition, 2012.